

인스타그램 릴스 트렌드 분석 자동화(Wrap-up 문서)

1. 프로젝트 개요

- 방대한 비정형 릴스 캡션 데이터에서 사용자의 질문에 맞춰 정보를 효율적으로 검색하고 특히 LLM의 '할루시네이션' 문제를 RAG 기반으로 해결하고자 함.
- 특히 한국어 데이터 감정분석에 특화된 접근을 시도하고 추천 릴스를 사용자에게 제공함으로써 필요로 한 데이터 분석이 가능하도록 구현하였으며, RAG를 이용하여 수집된 데이터 내에서 사용자의 질의를 해결하고자 구현함.

2. 핵심 구조

- 자동화된 데이터 수집

Apify 및 Instaloader를 활용하여 공식 API에 의존하지 않고 특정 해시태그 릴스 캡션 및 관련 메타데이터를 효율적으로 수집.

- 정교한 텍스트 분석

수집된 한국어 텍스트를 Konlpy를 통해 형태소 분석하여 정제하고, Hugging Face Transformers 모델(예: snunlp/KR-FinBert, SBERT, KeyBERT)을 사용해 감정 분석과 핵심 키워드 추출.

- 질문 기반 요약 시스템 (RAG) 구현

LangChain, UpstageEmbeddings, FAISS(VectorDB), ChatUpstage(Solar API)를 결합한 RAG 구조를 핵심으로 하여, 수집된 릴스 내용 기반으로 사용자 질문에 정확하고 관련성 높은 답변을 제공하는 멀티턴 대화형 시스템을 구축

- 시각화 대시보드

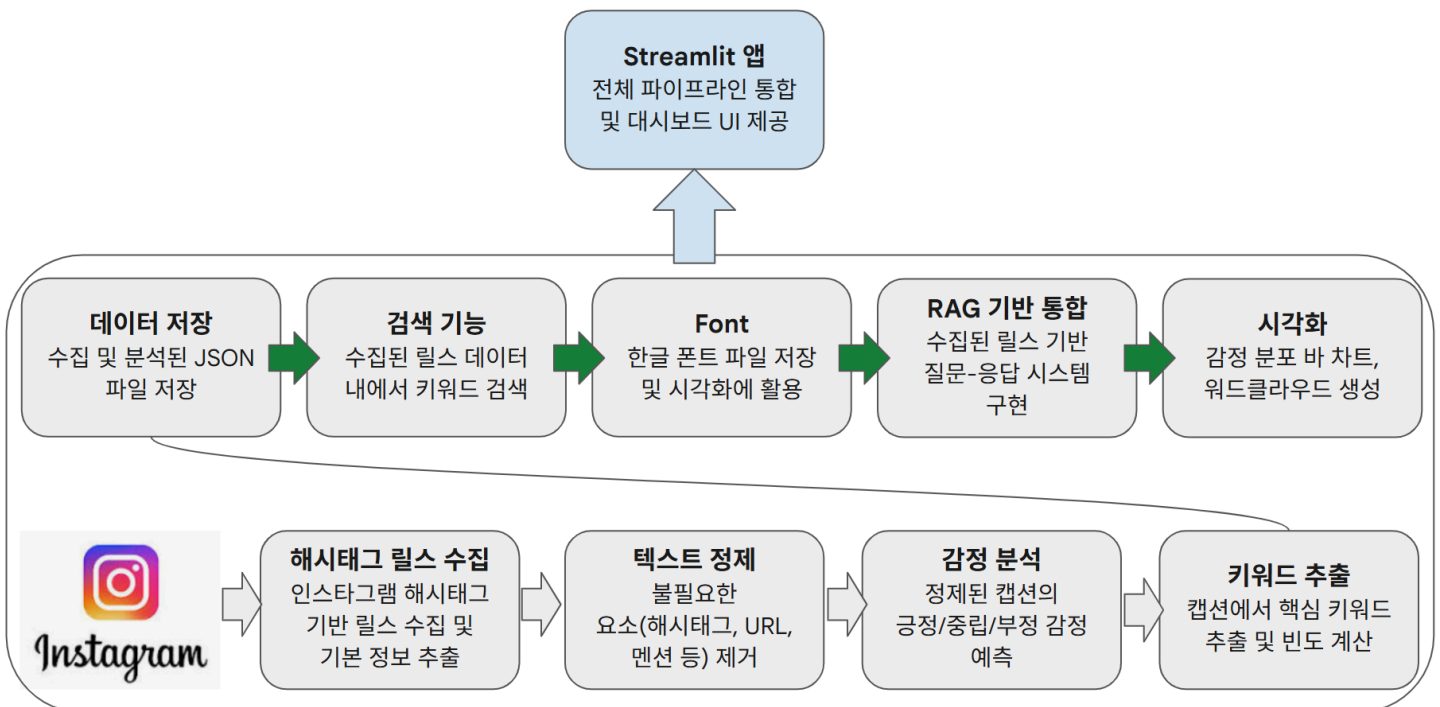
Streamlit 웹 애플리케이션을 통해 분석 결과(감정 분포 그래프, 워드클라우드 등)를 직관적으로 시각화하여 제공하며, 사용자 친화적인 UI와 사용성 개선에 중점.

3. 효과(분석 결과 시각화 및 사용자 경험 개선)

- Streamlit 웹 애플리케이션을 통해 감정 분포 그래프, 워드클라우드, 관련 해시태그 제안, 인기 릴스 등의 분석 결과를 직관적으로 시각화하여 제공.
- 사용자 친화적인 UI 구현을 위해 탭 방식에서 한 화면 분할 방식으로 전환하여 사용자 경험을 개선하고 `st.session_state`를 활용하여 이전 입력 정보와 분석 결과가 유지되도록 하여 사용성을 향상.
- 시각화 결과의 직관성에 대한 정성적 평가(감정 분석)를 통해 사용자 관점을 제시.

4. 사용 기술 스택

- 사용 기술 스택 리스트
 - 데이터 수집: ApifyClient(instaloder 시도 후 차단됨)
 - 텍스트 전처리: re, pandas, Konlpy
 - 감정 분석: Hugging Face Transformers (beomi/kcbert-base)
 - 키워드 추출: KeyBERT, SentenceTransformer/SBERT
 - RAG & LLM: LangChain, UpstageEmbeddings, FAISS(VectorDB), ChatUpstage(Solar API)
 - 시각화: matplotlib, seaborn, wordcloud, Streamlit
 - 기타: Git, GitHub, .env, 폰트 처리



5. 프로젝트 회고

- 잘한 점

- **RAG** 시스템의 성공적인 구현

LLM의 답변 정확도와 관련성을 높이기 위해 **RAG** 구조를 적용하여 도메인 특화 질문에 대응할 수 있었음.

- 다양한 기술 스택 통합

웹 스크래핑, 텍스트 전처리, **NLP** 모델, 벡터 데이터베이스, **LLM**, 웹 프레임워크 등 여러 기술 스택을 성공적으로 통합하고 학습한 경험을 프로젝트 구현에 적용해볼 수 있었으며, 수집, 정제, 감정분석, 키워드 추출 등 멀티턴 **RAG** 시스템의 작동을 위한 기반이 됨.

- 사용자 친화적 **UI** 및 한국어 특화 대응

- 아쉬운 점

- 정량적 평가 부재: 감정 분석이나 키워드 추출 모델의 성능을 정량적으로 평가할 수 있는 명확한 기준이나 데이터셋을 마련하지 못하여 객관적인 성능 지표 부족
- **LLM** 컨텍스트 윈도우 관리 미흡: 멀티턴 대화에서 **LLM**의 컨텍스트 제한에 대한 명시적인 관리 로직이 부재 하고, 핵심 기능에서의 개선 필요성을 인지
- **Apify API**의 무료 티어 제한. (토큰 사용량 체크, **api** 활용량 분석 정보가 필요했었다. 등)
- 복잡한**UI**, **FAQ**, 문서 내려받기 등 사용자 친화적 기능 부족

- 개선 계획

- **RAG** 시스템 심화: **LLM** 컨텍스트 관리 로직 추가(**chat_history** 길이 제한 또는 대화 요약 기술 적용)
- 테스트 데이터, 어떤 지표로 평가할 것인가에 대한 기준 필요.

6. 총평

- 파이썬 핵심 프로그래밍 개념(자료구조, 함수, 모듈/클래스, 예외 처리 등)과 **Hugging Face pipeline**을 이용한 **AI** 태스크 처리(**text-generation**, 감성 분류 등) 경험.
- **Streamlit**을 통한 웹 **UI** 구축, **LangChain**을 이용한 **RAG** 시스템 구현 과정에서 얻은 학습 경험.
- 멀티턴 대화 시 문맥 유지 능력을 고려한 **LLM** 활용은 구현이 잘 되었으나, 멀티턴의 사용횟수, 프롬프트 엔지니어링에 대한 기술적 반영이 구현되어있지 않았고, 욕설이나 성적 표현과 같이 불필요한 언어의 필터링 부분 미적용.
- 최초 구상했던 부분과 달리 현실적인 한계점과 미처 고려하지 못했던 맹점이 프로젝트 진행 시 발견됨에 따라 이 프로젝트를 대신하여 새로운 프로젝트를 진행해야할까라는 현실적인 고민

